ISCD Homework 2

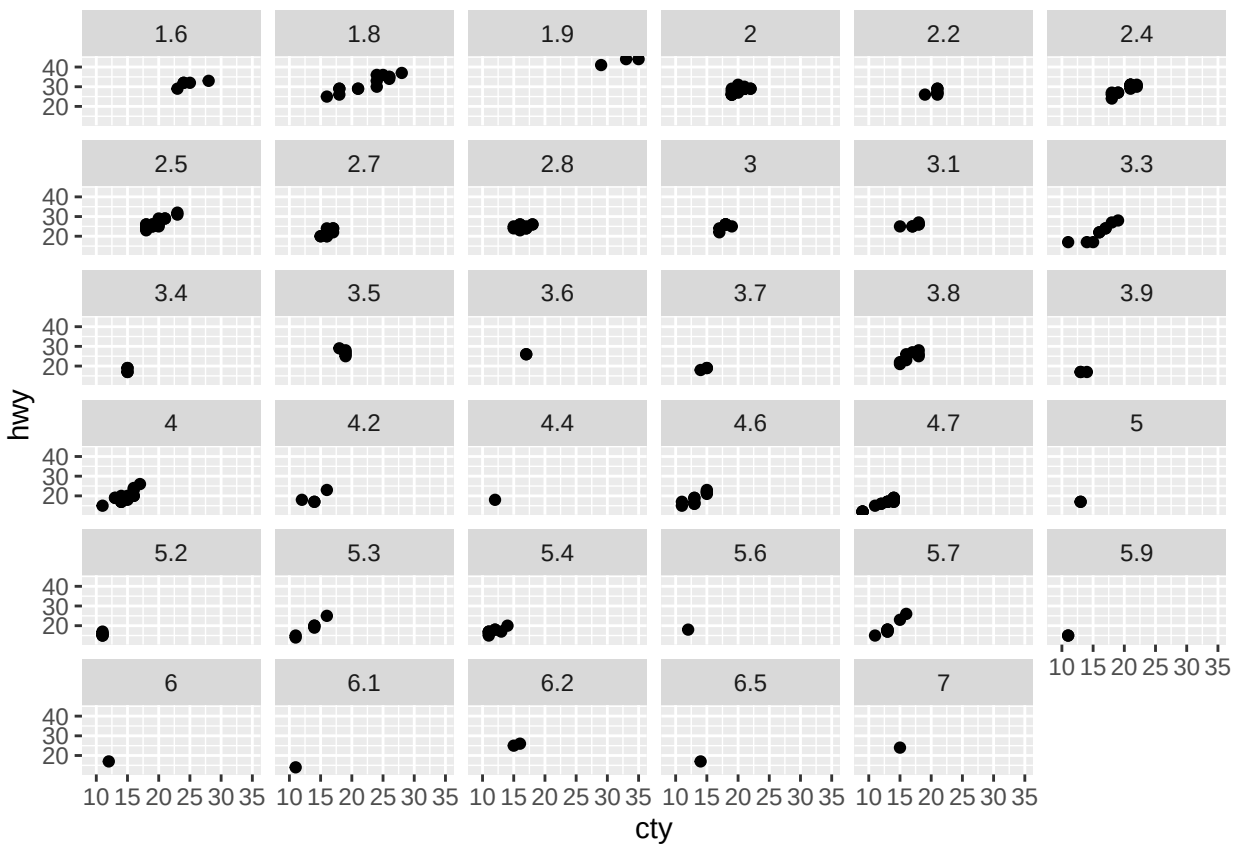
Anurag Bhunia(bmat2508@isibang.ac.in)

Question 1

a)

1.

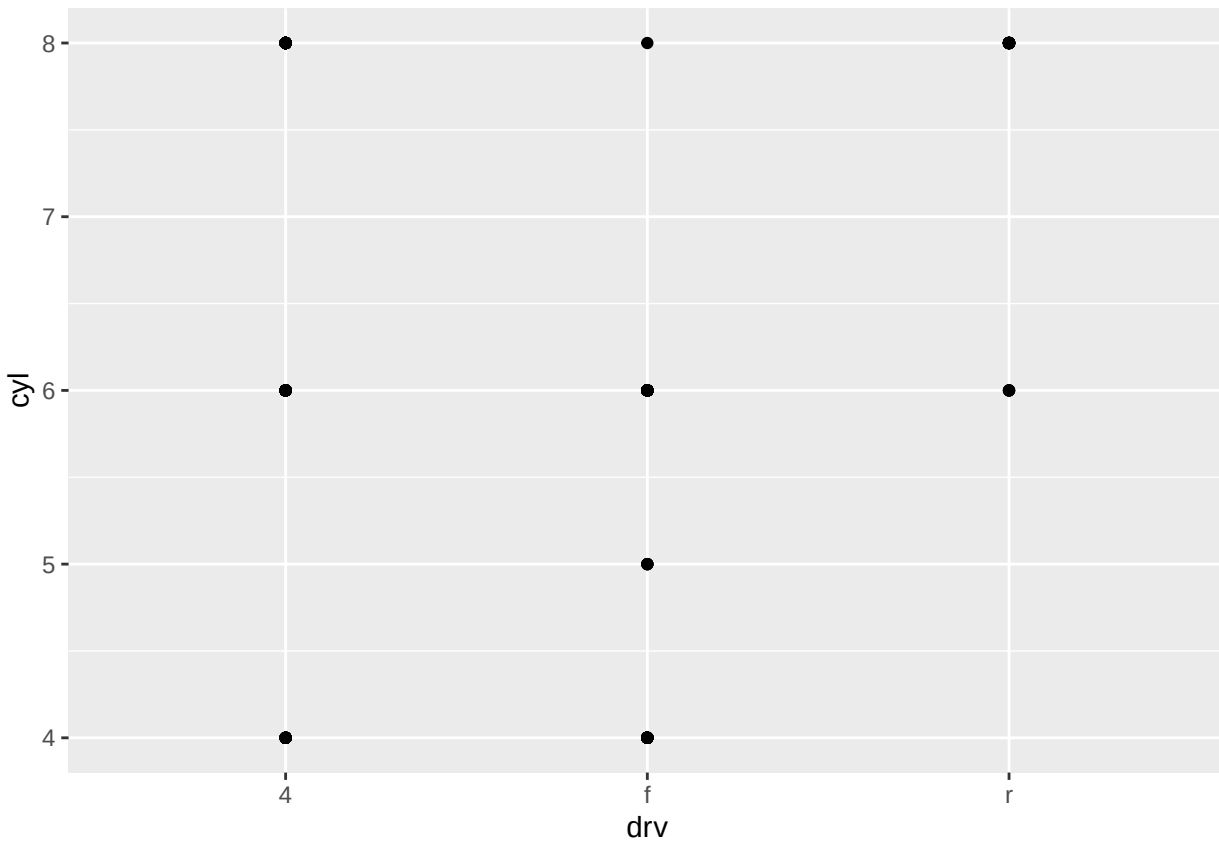
```
ggplot(mpg,aes(cty, hwy))+geom_point()+facet_wrap(~ displ)
```



In the dataset **mpg**, displ (which accounts for the engine displacement of a vehicle) is a continuous variable. Faceting on it will result in treating each numerical value as a category of its own, possibly resulting in huge number of blocks upon faceting. This is the general issue when faceting directly on a continuous variable.

2. The empty cells indicate that there are no cars with the specific wheel and cylinder type configuration. For example, the cell denoting 4-wheeled vehicles having 5 cylinders is empty because there are no such vehicles.

```
ggplot(mpg)+geom_point(mapping=aes(x=drv,y=cyl))
```



In this plot, we can see that that whatever cell combination was empty, if we go along that specific wheel type then they do not have points corresponding to those many cylinders, again denoting absence of such type of vehicles.

3. In the first case, we have no column specification in the faceting, so it creates 3 copies of the range of the hwy variable and shows the engine displacement vs. highway distance seperately for each of the 3 wheel types. In the second case, again since no row is specified, it just shows the engine displacement vs. highway distance for each of the 4 cylinder types.

4. Faceting would be particularly useful if we want to view the data into localised copies, which allows us to observe trends in smaller categories of the dataset. The colour aesthetic doesn't offer the same advantages quite as well. On the other hand, having a very compartmentalized view at the dataset may not help to look at the bigger picture. Certain global trends are more noticeable when using the colour aesthetic. Also since looks at the dataset in more detail, we lose the compactness that the colour aesthetic provides. For a larger datatset, faceting would prove more useful, since colour would have very little significance if one had to compare among numerous records, while faceting would help to divide the dataset into smaller, more manageable chunks.

5. nrow controls the number of rows that will be formed upon faceting. Suppose, we facet on variable "class". There are a total of 7 vehicle classes in mpg. Setting nrow=2 will result in 2 rows of cells with one having 4 and the other having 3. Similarly ncol controls how many columns will be formed. The other options that control the layout of panels include: scales(determines the range of values for each of the panels; they can either be fixed or free for each of the panels), dir(determines the directions in which successive panels appear; for example if we want them to grow vertically, we set dir="v" and for horizontal growth we set dir="h"), as.table(determines how the panels will be laid out; if as.table=TRUE then the panels will be laid out like a table in which case the panels start from the top-left corner, if its set to FALSE then they will be laid out like a mathematical graph instead and in this case the panels start from the bottom-left corner), drop(if any factor level has no observations from our given dataset then it is good enough to drop it all together, by default drop is set to TRUE),

strip.position(this controls the placement of labels on the facets; bu default it is "top", but we could also change it, for example to "bottom") and shrink(setting this to TRUE will result in the limiting of the axis range based on the statistics while setting it to FALSE would set the range as per the raw data).

facet_grid doesn't have the nrow and ncol arguments because there the data is represented in a matrix-like layout; if the row variable has 3 levels and the column variable has 4 levels then there can only be 12 facets based on any combination of levels from each of these variables.

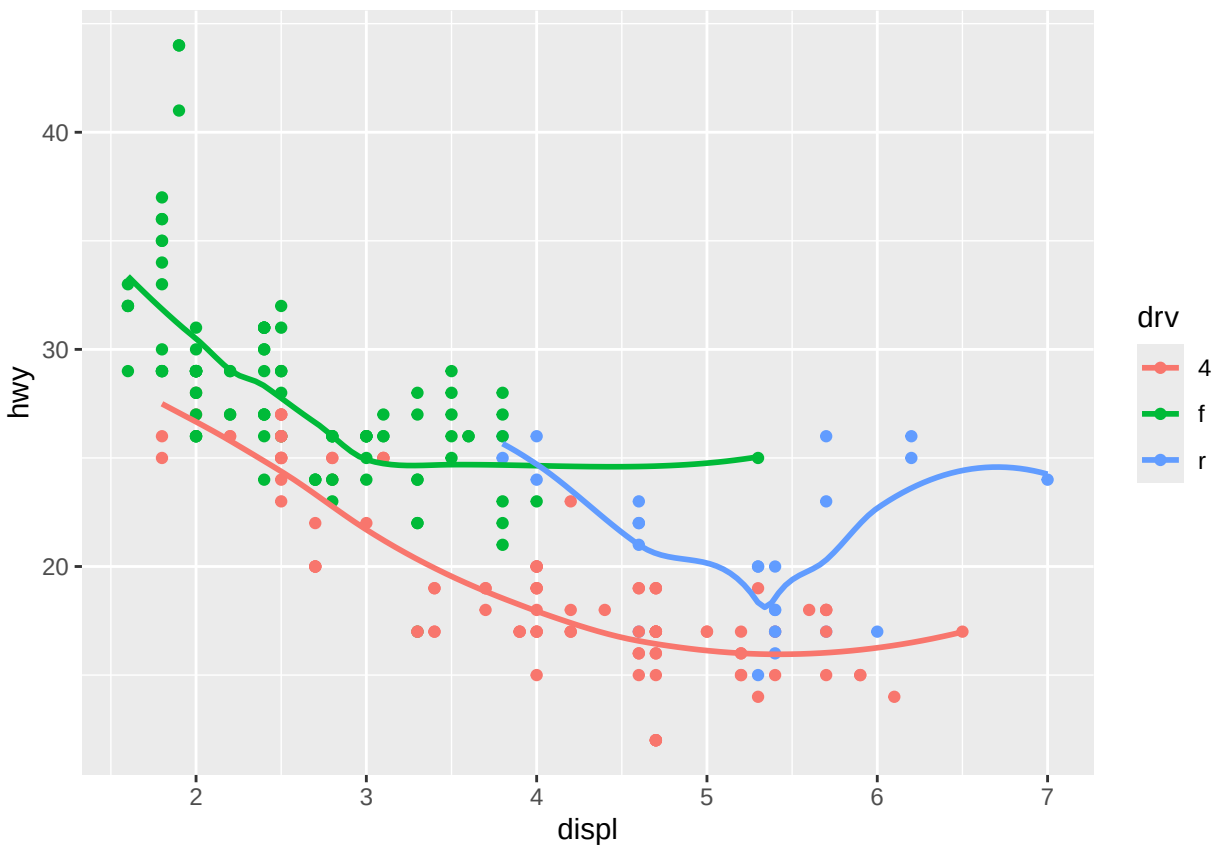
6. It is usually better to set the variable with more unique levels to the columns because of visual convinience. Screens are much wider column-wise and thus our visual pattern is adjusted to the same; having more levels column-wise would help to scan through the data more efficiently.

b)

1. In order to draw a line chart, one would usually use the geom geom_line.
2.

```
ggplot(mpg,mapping=aes(x=displ,y=hwy,colour=drv))+geom_point()+geom_smooth(se=FALSE)
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



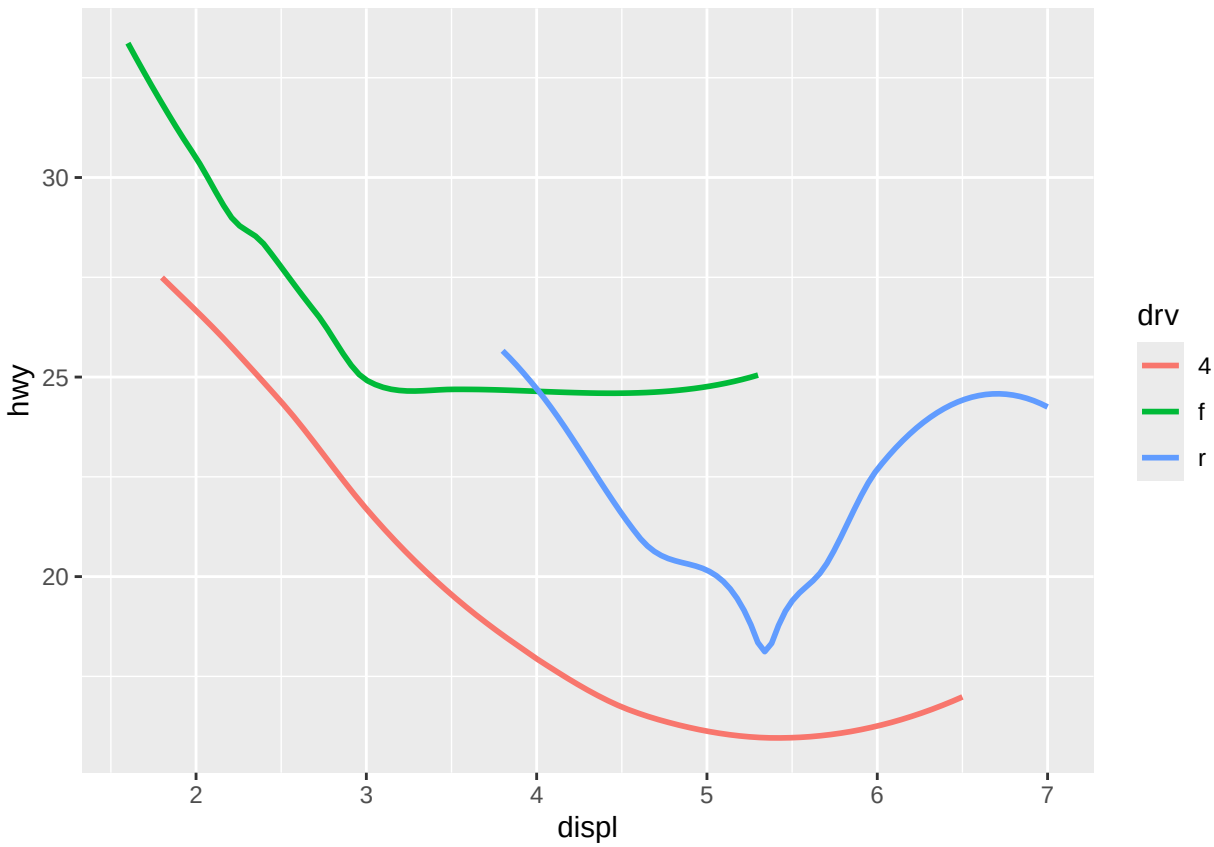
As expected, this plot had a global mapping defined that overrides the default specifications of the geom_point and geom_smooth. Normally only one line would have been drawn under the geom_smooth with the confidence interval being shaded. However as the global property has been defined to look at the 3 drive trains individually, the smooth plot obeys that rule. Also as a local mapping, the confidence interval is not shown.

3. show.legend=FALSE makes it so that the legend is no longer shown on the side of the plot, thus not denoting what each individual colour or shape is supposed to indicate. If we remove this line, ggplot

will by default include the legend wherever appropriate. Here is an example of the same:

```
ggplot(mpg)+geom_smooth(mapping=aes(x=displ,y=hwy,colour=drv),se=FALSE)
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



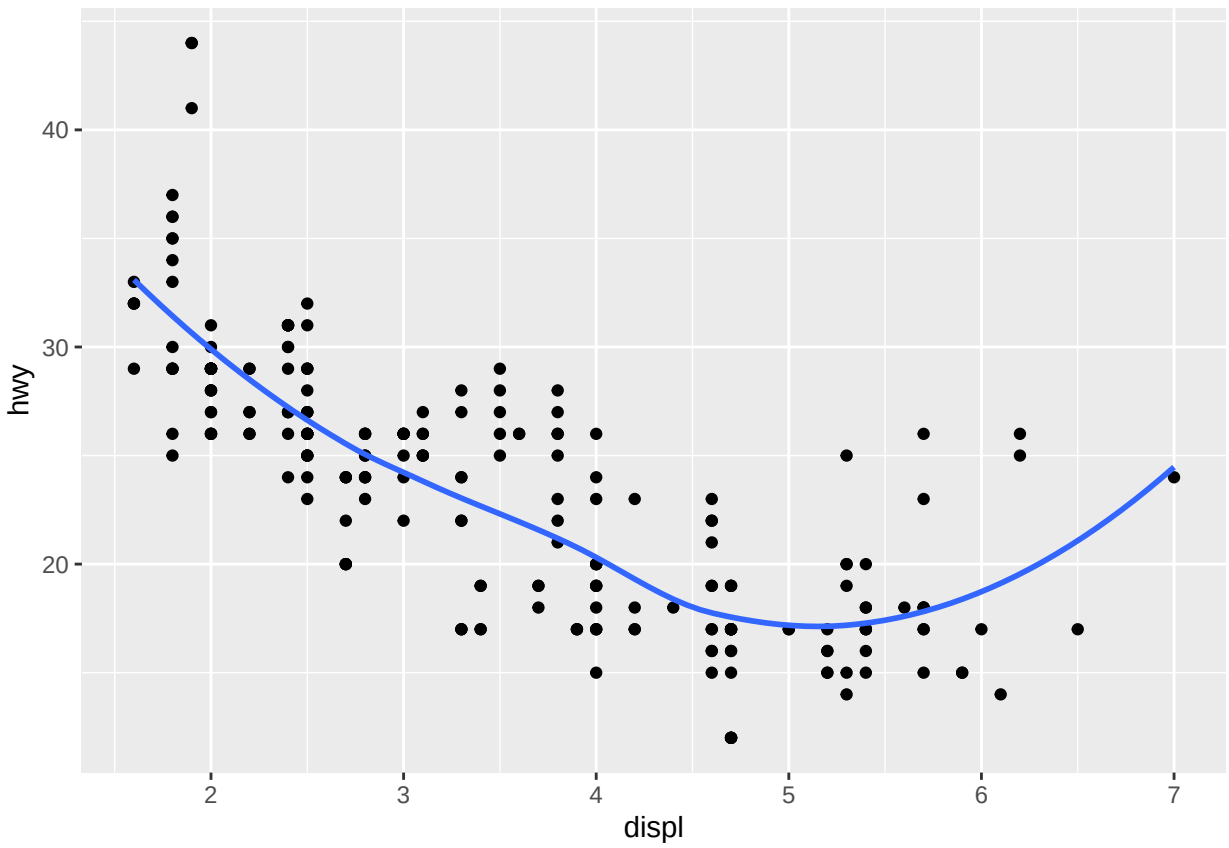
4. The se argument is used to show the confidence interval around the line by calculating the standard error and using that to create a region whose boundaries denote some constant multiple of the standard error(typically within ± 1.96 about the mean line).

5. No, these two plots will be exactly the same; in the first one we define the global mapping within ggplot() and hence they apply to both the scatter and the smooth plot. In the second one, we do not define anythig inside ggplot but define the same universal properties individually to each of the plots.

6.

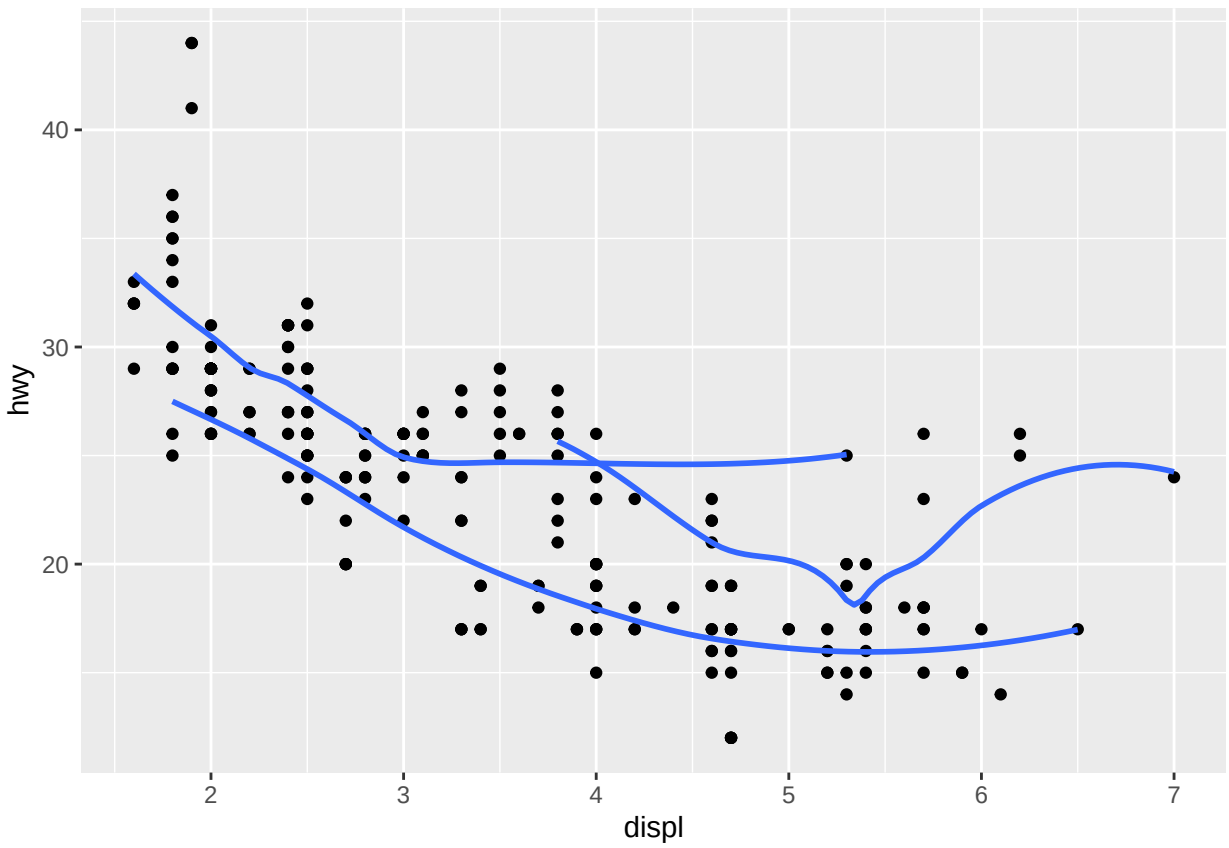
```
ggplot(mpg,mapping=aes(x=displ,y=hwy))+geom_point()+geom_smooth(se=FALSE)
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



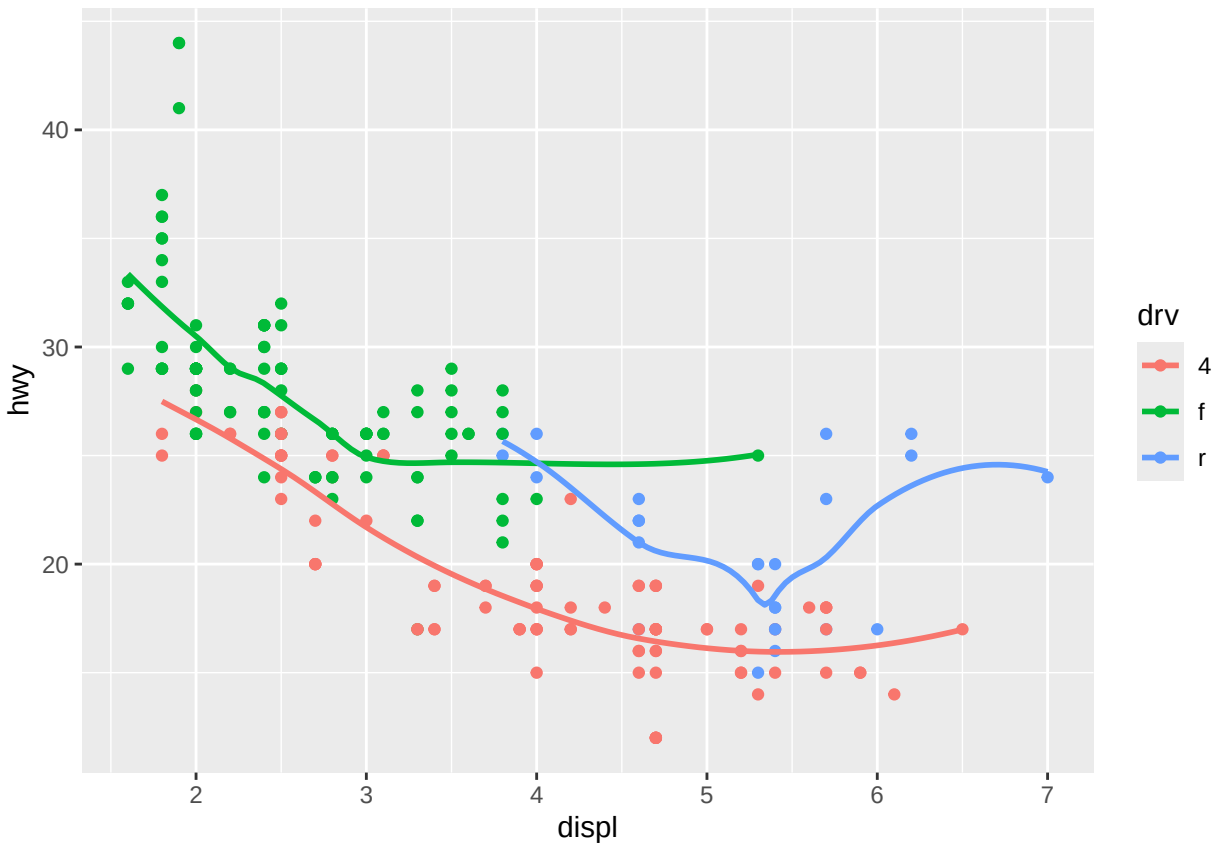

```r
ggplot(mpg,mapping=aes(x=displ,y=hwy))+geom_point()+geom_smooth(mapping=aes(group=drv),se=FALSE)
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



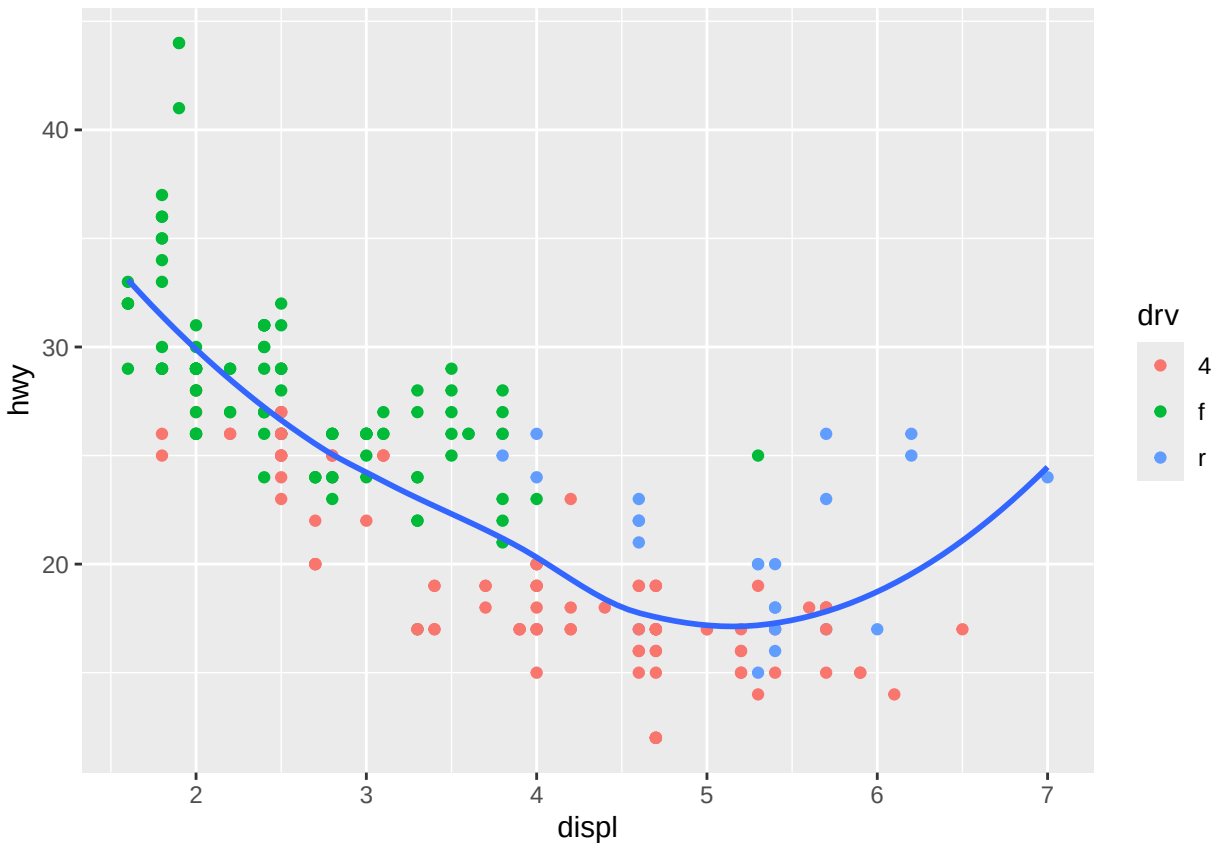

```r
ggplot(mpg,mapping=aes(x=displ,y=hwy,colour=drv))+geom_point()+geom_smooth(se=FALSE)
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



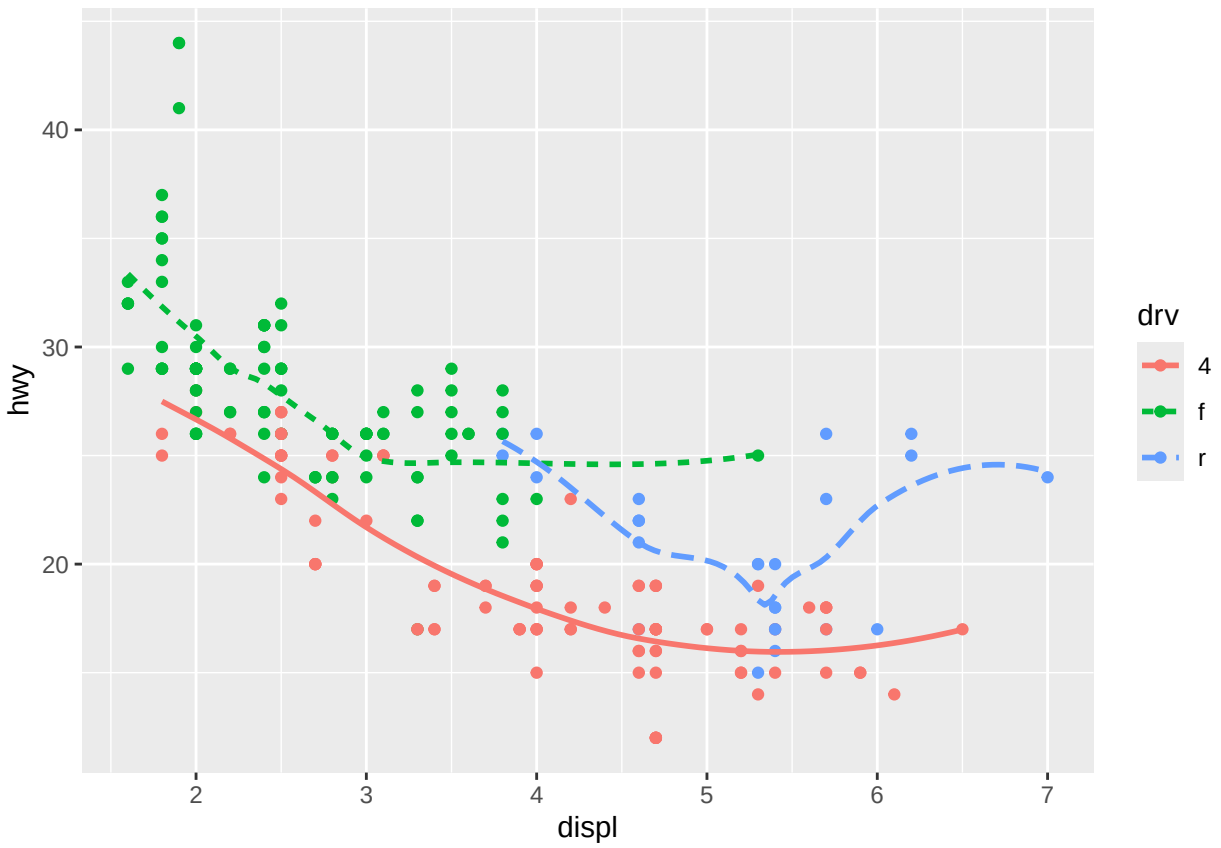

```
ggplot(mpg)+geom_point(mapping=aes(x=displ,y=hwy,colour=drv))+geom_smooth(mapping=aes(x=displ,y=hwy),se=
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

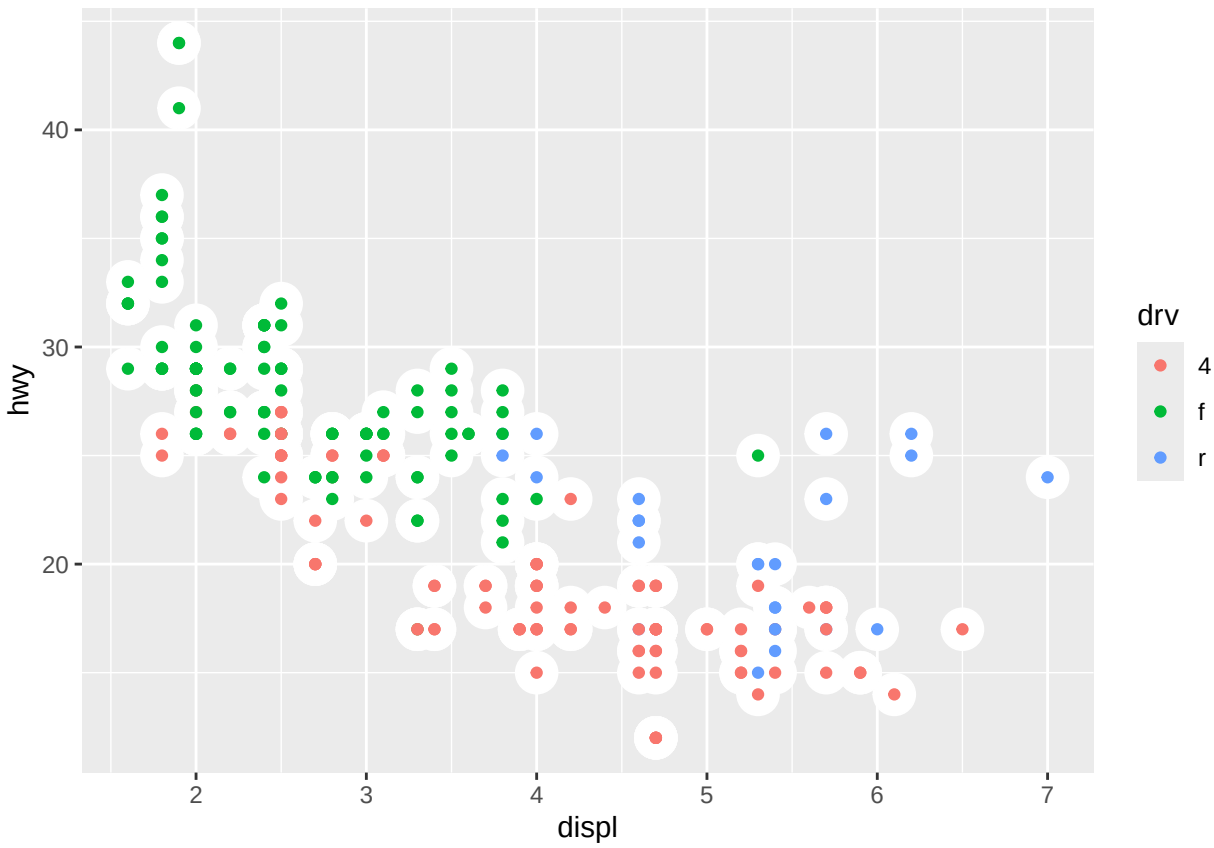



```
ggplot(mpg,aes(x=displ,y=hwy,colour=drv))+geom_point()+geom_smooth(aes(linetype=drv),se=FALSE)
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



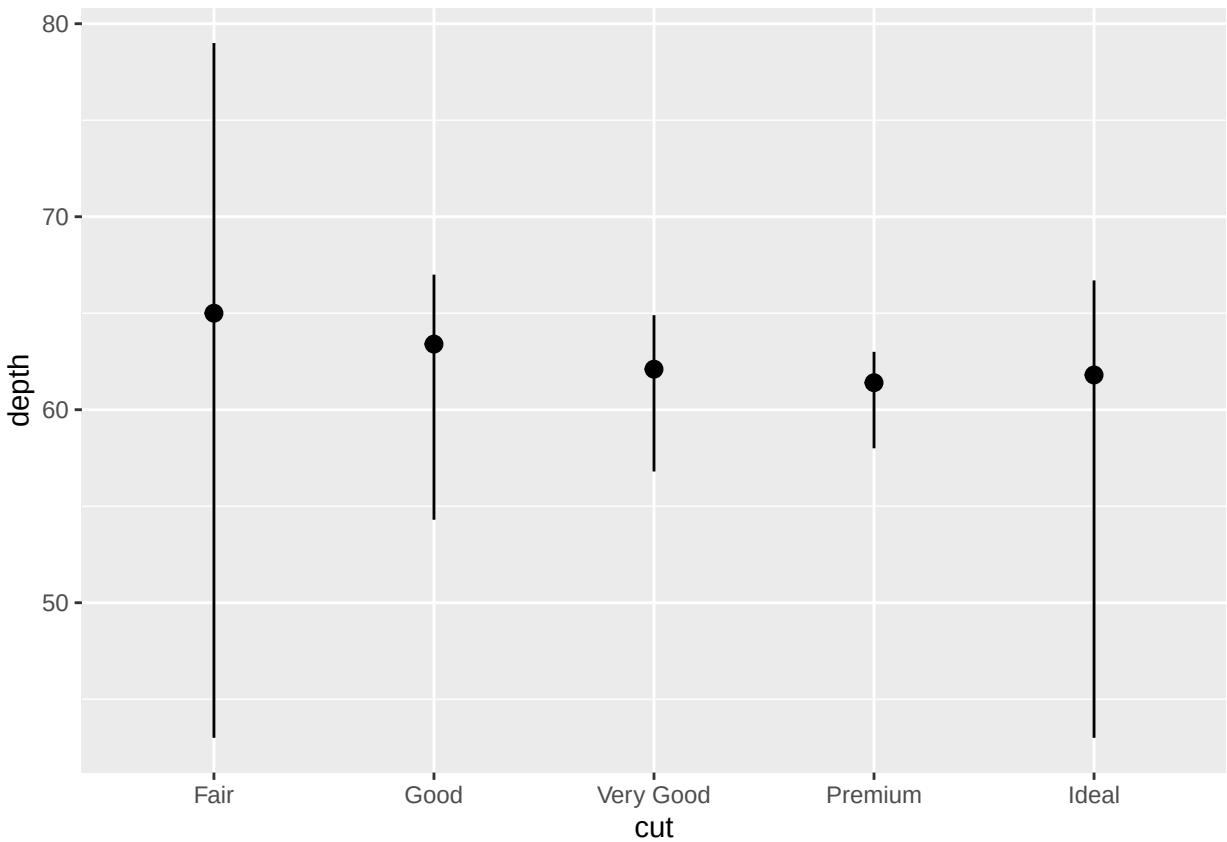

```
ggplot(mpg,aes(x=displ,y=hwy,colour=drv))+geom_point(size=7,colour="white")+geom_point()
```



c)

1. The default geom associated with stat_summary is pointrange. We could have rewritten the above code using geom_pointrange like so:

```
ggplot(diamonds,aes(x=cut,y=depth))+geom_pointrange(stat="summary",fun.min=min,fun.max=max,fun=median)
```



2. Both geom_bar and geom_col create bar charts but they differ in the way they plot the given data: geom_bar() looks at raw data, computes the frequency of each variable and then draws the bars for them. Thus the default stat is stat_count(). On the other hand, geom_col does not do any computation; it is best suited when attributes like count of a given variable are already available, it simply draws a bar tall enough to denote that. Thus, it uses the stat stat_identity().

3. The following is a list of all available geom-stat pairs:

i. geom_bar(), stat_count()

ii. geom_col(), stat_identity()

iii. geom_histogram(), stat_bin()

iv. geom_point, stat_identity()

v. geom_boxplot(), stat_boxplot()

vi. geom_density, stat_density()

vii. geom_smooth(), stat_smooth()

viii. geom_pointrange(), stat_summary()

ix. geom_violin(), stat_ydensity()

x. geom_contour(), stat_contour()

xi. geom_hex(), stat_bin_hex()

xii. geom_quantile(), stat_quantile()

xiii. geom_density_2d(), stat_density_2d()

xiv. geom_crossbar(), stat_summary()

xv. geom_errorbar(), stat_summary()

xvi. geom_linerange(), stat_summary()

xvii. geom_freqpoly(), stat_bin()

xviii. geom_dotplot(), stat_bindot()

xix. geom_area(), stat_align()

xx. geom_line(), stat_identity

These pairs all have the same aesthetic mapping requirements.

4. The variables computed by stat_smooth are:

i. after_stat(y): Predicted value of y at a given x.

ii. after_stat(ymin): The infimum of the confidence interval.

iii. after_stat(ymax): The supremum of the confidence interval.

iv. after_stat(se): The standard error of the prediction.

The parameters that control its behavior are:

i. method: The method of predicting future trends in data.

ii. formula: This tells the chosen model how to relate the y variable to the x variable.

iii. se: This computes the standard error and also generates the confidence band around the mean line.

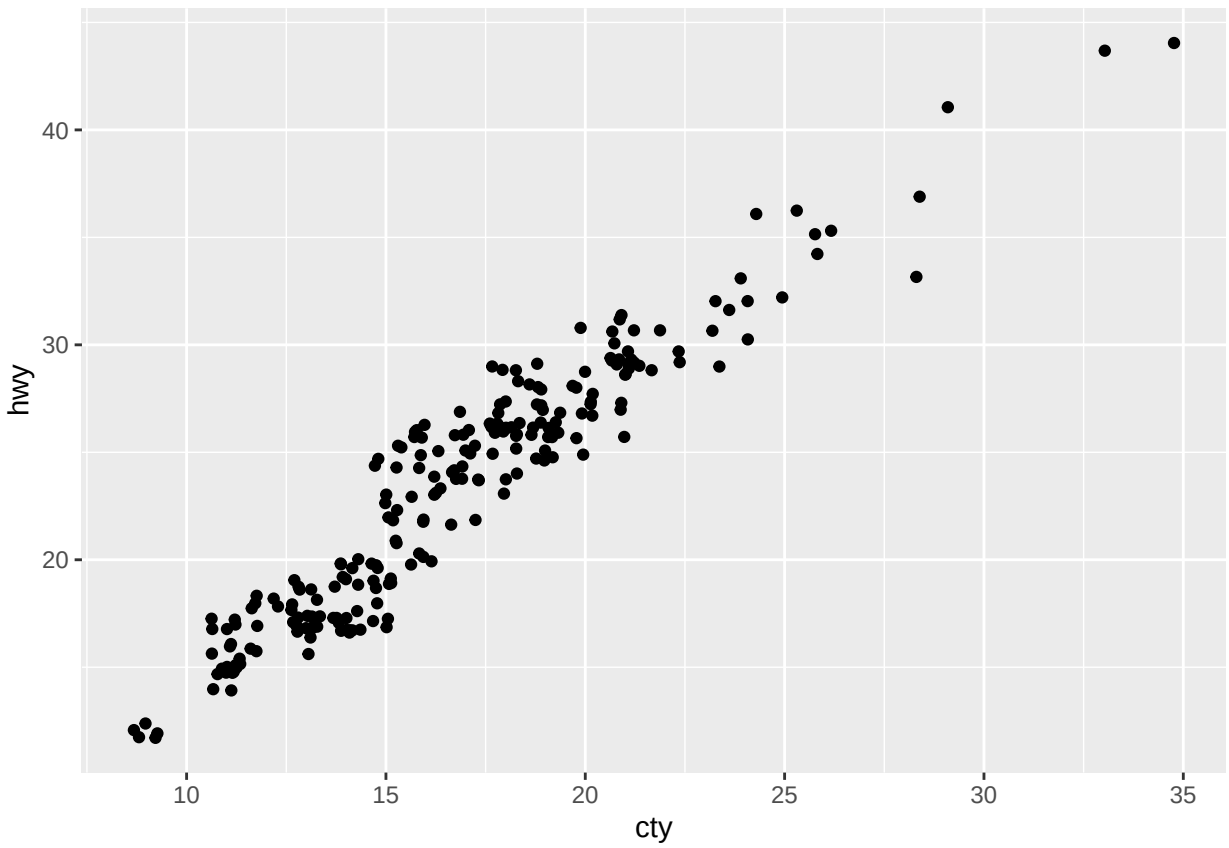
iv. show.legend: As seen in questions above, setting this to FALSE would not generate a legend for our plot. By default, it is always set to TRUE.

5. The problem with the first graph is that by default each level of a variable is treated as a group of its own. So, it amounts to asking "what proportion of fair diamonds is fair?". This is why all bars have the same height. In the second graph, the height remains the same due to the same issue and instead each bar is shaded by a range of colors denoting the colour of dimaonds for that level of cut. While that offers a look at the color distribution for the various cuts, it does not fix our issue at hand. When we set group=1(or any arbitrary group number) we instruct ggplot to treat every category as a group of its own. This results in the fair diamonds being counted amongst all other type of cut of diamonds, as required.

d)

1. In this example we have the classic problem wherein the datatset mpg has a total of 234 points but only shows about 73 of them. This is due to overplotting and can be solved by setting position="jitter" which adds some amount of random noise to each point which eventually spaces them out since it is unlikely that two distinct points would have the same amount of random noise. Thus we make the following adjustment:

```
ggplot(mpg)+geom_jitter(aes(x=cty,y=hwy))
```

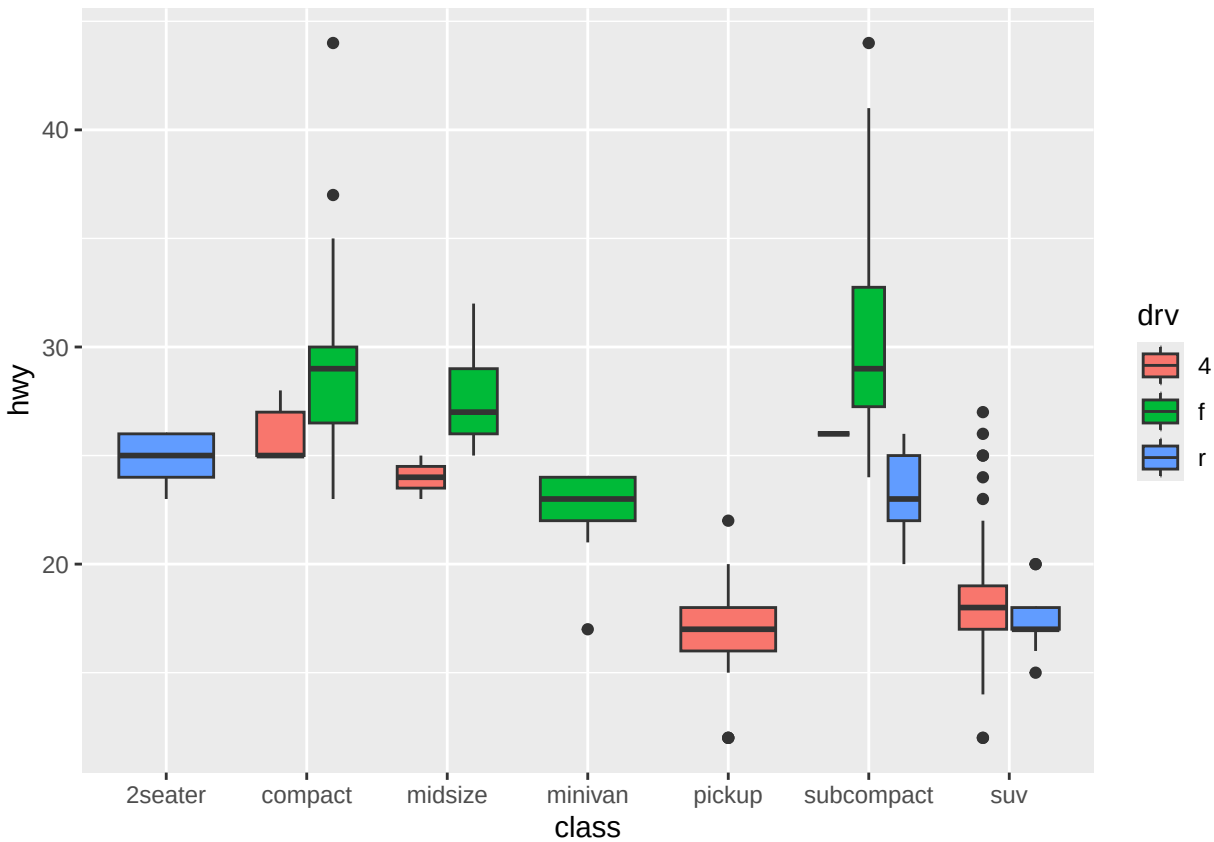


2. geom_jitter() normally adds random noise in both horizontal and vertical directions. The two parameters height and width help to steer the jittering in a given direction by the specified amount.

3. While both geom_jitter() and geom_count() are used to tackle overplotting, there are some differences in their functionality. Firstly, geom_jitter() adds random noise around a given point, while geom_count() resizes points based on the number of overlaps. Secondly, geom_jitter() uses the stat stat_identity, while the latter uses the stat stat_sum(). Finally, while geom_jitter sacrifices spatial accuracy, geom_count() makes it so that each point is in its own spot.

4. The default position adjustment for geom_boxplot is dodge. Here is an example that demonstrates it:

```
ggplot(mpg,aes(x=class,y=hwy,fill=drv))+geom_boxplot()
```

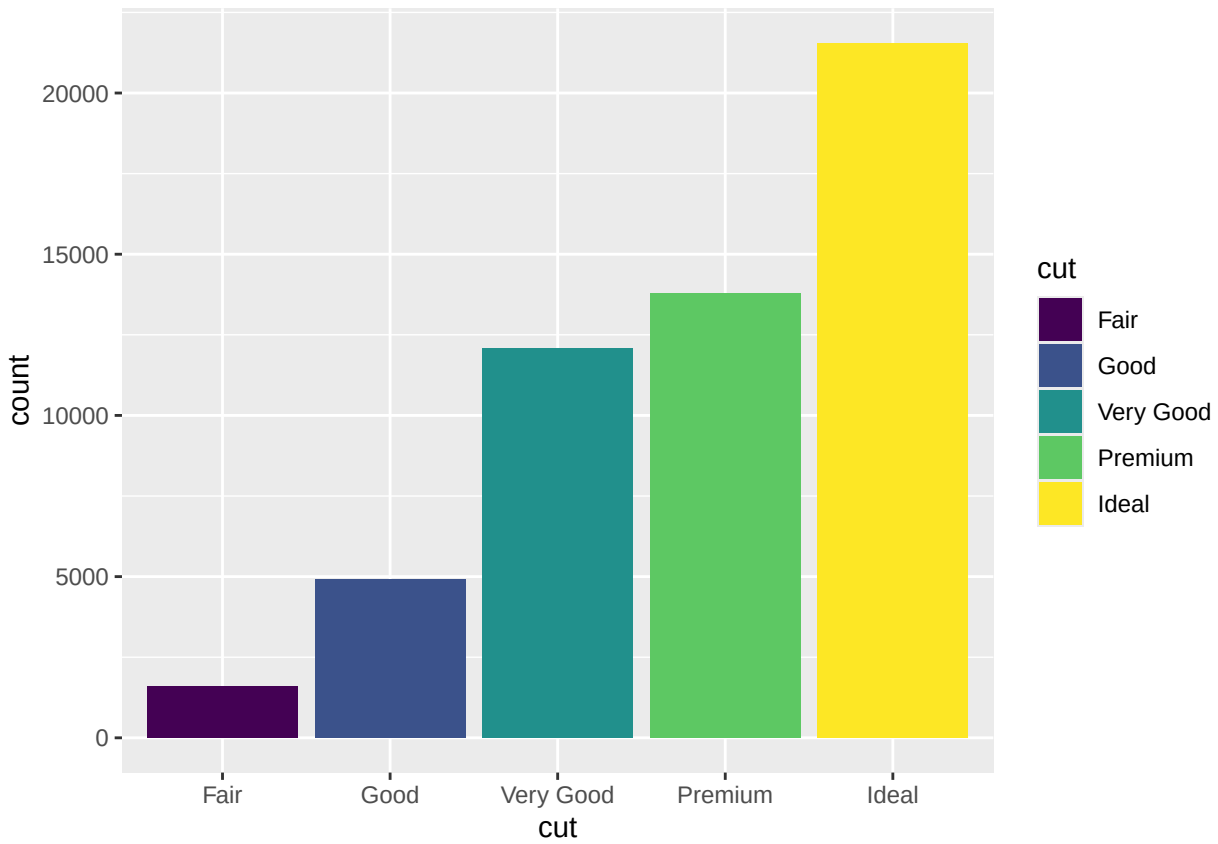


As we can see here, instead of stacking the boxes for different drive trains, it plots them seperately side by side.

e)

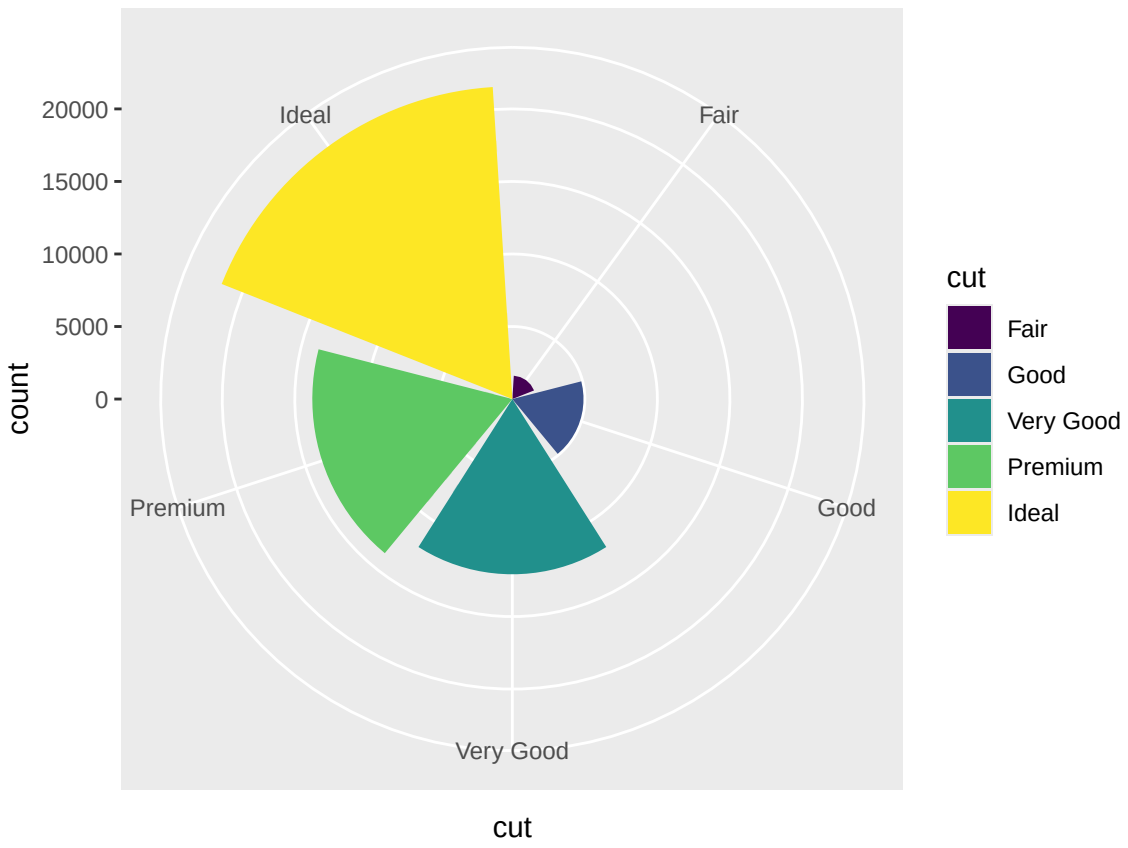
1. Consider the following example which turns a stacked bar graph into a pie chart:

```
ggplot(diamonds,aes(x=cut,fill=cut))+geom_bar()
```



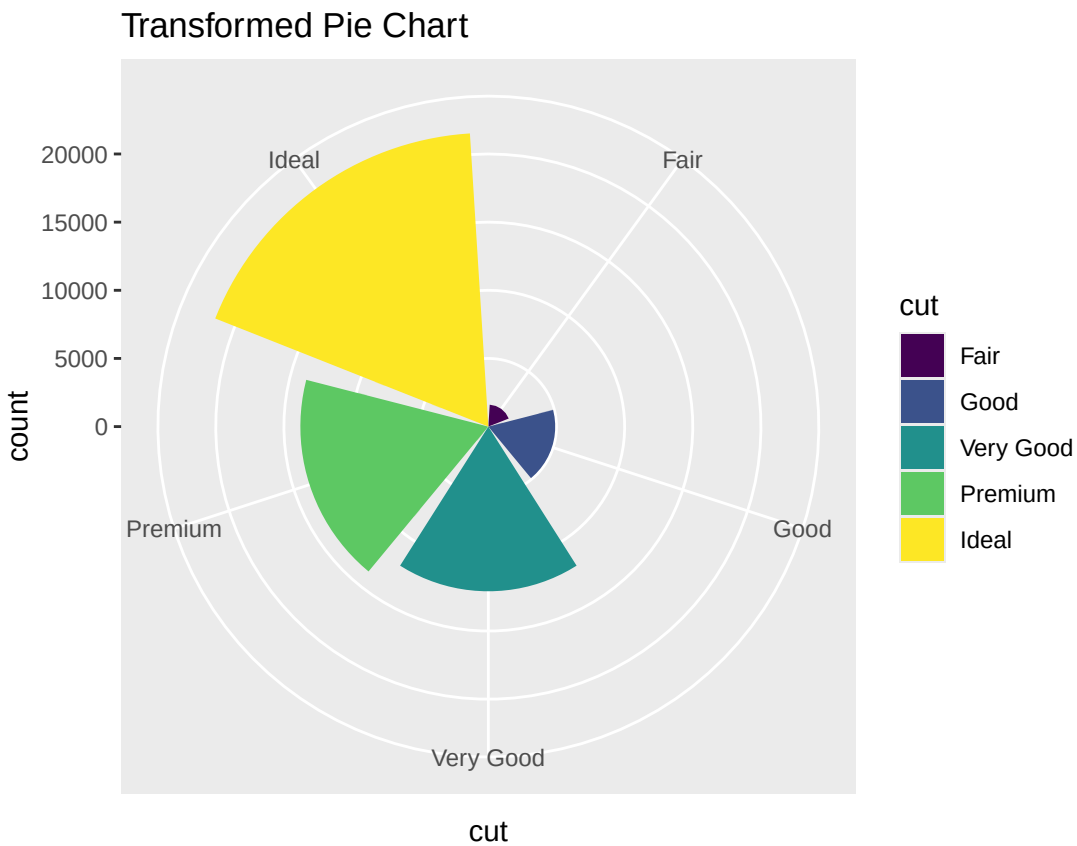
From this we go to a pie chart:

```
ggplot(diamonds,aes(x=cut,fill=cut))+geom_bar()+coord_polar()
```



2. The labs() function lets us create custom plot title,subtitle,x label,y label,caption and so on. The following example illustrates this:

```
ggplot(diamonds,aes(x=cut,fill=cut))+geom_bar()+coord_polar()+labs(title="Transformed Pie Chart")
```



3. While both coord_quickmap() and coord_map() create maps, there is a difference in quality. coord_map() uses the mapproj package to create detailed and highly accurate projection of points on flat map and thus it takes some time to do so. coord_quickmap on the other hand creates maps rather quickly, which it does by simply fixing the aspect ratio of the plot, rather than re-projecting each point.

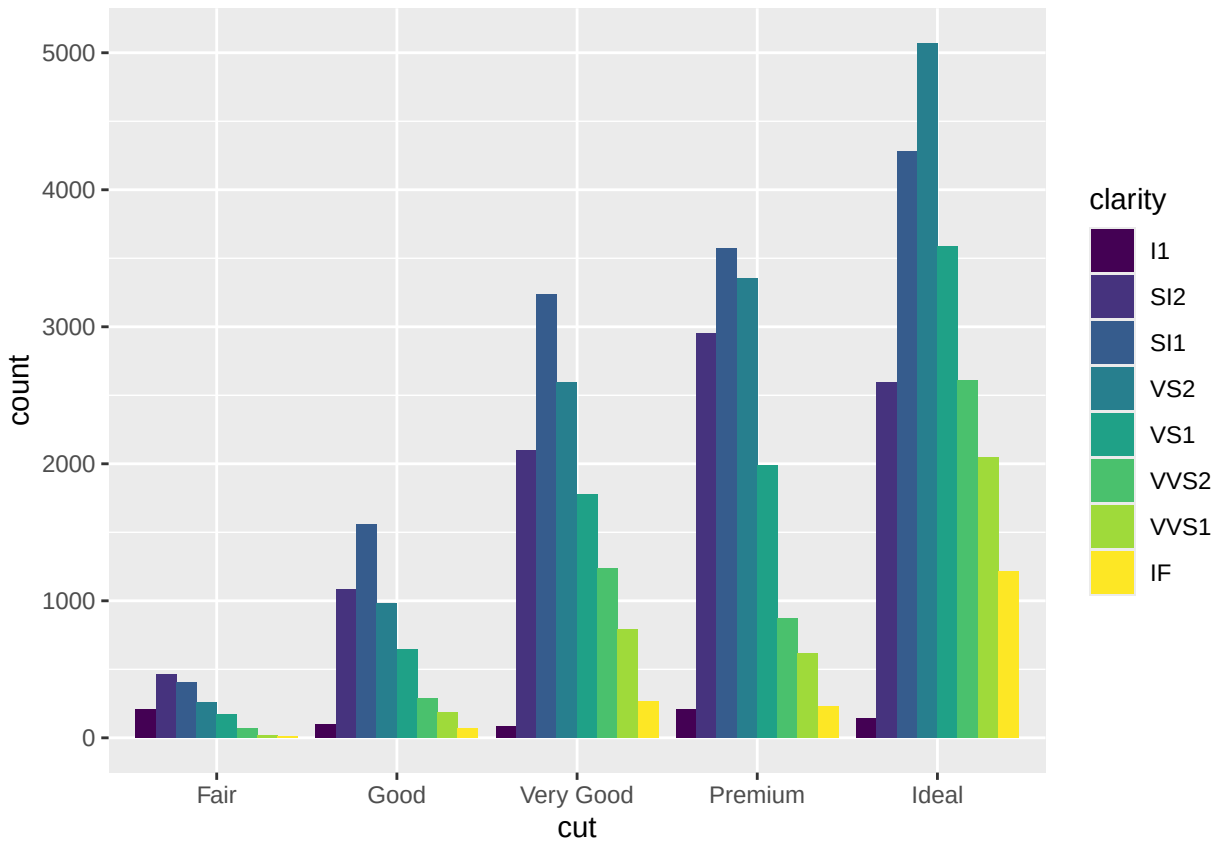
4. This plot tells us that as the city mileage increases, the highway mileage increases as well. In fact, the highway mileage consistantly stays ahead of the city mileage. By default, ggplot resizes the aspect ratio of a plot to fill the available space in the screen. coord_fix() helps to maintain the same aspect ratio. In this case, geom_abline adds a reference line to the plot. It represents the scenario where the highway mileage would be equal to the city mileage.

Question 2

a) The diamonds dataset, included in ggplot, contains prices and other attributes like carat,colour,cut and clarity of nearly 54000 diamonds.

b)Here is a code to generate the given plot:

```
ggplot(diamonds)+geom_bar(aes(x=cut,fill=clarity),position="dodge")
```



c) From the graph, it is easily seen that there are more diamonds available which are of higher quality. Also, almost across all categories, dimaonds of clarity I1 were the fewest, whereas those of clarity SI1 and VS2 were consistently more in number than the rest.